Code-Layout, Readability and Reusability

Date	08/07/ 2026
Team ID	
Project Name	Human Development Index Predictor
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the HDI Predictor codebase in terms of its structure, readability, and reusability. The project is built entirely in HTML5, CSS3, and Vanilla JavaScript. Well-organised code with proper naming conventions, inline comments, and modular function design ensures the tool is maintainable and can be extended — for example, to add multi-country comparison, UNDP API integration, or CSV/PDF export — without reworking existing computation logic.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	**Consistent Indentation**	Uniform 2-space indentation used throughout all JS, HTML, and CSS files	Yes	VS Code Prettier formatter enforced; all slider event handlers and computation functions are consistently indented
2	**Proper File Structure**	Files and folders are logically organised	Yes	index.html (markup), style.css (presentation), hdi.js (computation logic), recs.js (recommendation engine) — clearly separated concerns
3	**Meaningful Variable Names**	Variables reflect their purpose clearly	Yes	Names like lifeExpectancyIndex, educationIndex, gniIndex, compositeHDI, tierClassification are self-documenting and follow camelCase convention
4	**Function / Method Names**	Functions are descriptively named	Yes	computeLifeIndex(), computeEducationIndex(), computeGNIIndex(), calculateHDI(), classifyTier(), generateRecommendations(), loadScenario() — all clearly express their purpose
5	**Code Comments**	Inline and block comments explain logic	Yes	Block comments above each function describe inputs/outputs; inline comments explain the UNDP geometric mean formula, goalpost constants, and tier boundary thresholds
6	**Modular Design**	Code is split into reusable functions/modules	Yes	HDI computation, tier classification, recommendation engine, and UI rendering are each isolated functions. computeLifeIndex() and classifyTier() are reused across scenario loading and live slider events
7	**No Redundant Code**	Duplicate or unused code is removed	Yes	A single update() function drives all slider changes; scenario loading reuses the same update() pipeline — no duplicated calculation logic across event handlers
8	**Error Handling**	Exceptions and errors are handled gracefully	Yes	Input validation clips all slider values to UNDP goalpost bounds using Math.min/Math.max; Math.cbrt guards against zero-index edge cases; GNI log(0) is protected by a minimum floor value of 100

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	**computeLifeIndex()**	JavaScript	Called by update() on every slider event and by loadScenario() for all 3 pre-built scenarios	High
2	**computeEducationIndex()**	JavaScript	Called by update() and loadScenario(); combines MYS and EYS sub-indices into a single education dimension score	High
3	**computeGNIIndex()**	JavaScript	Called by update() and loadScenario(); normalises GNI per capita using natural log scaling per UNDP methodology	High
4	**classifyTier()**	JavaScript	Used by update() and loadScenario(); maps any HDI score to Very High / High / Medium / Low and returns colour theme	High
5	**generateRecommendations()**	JavaScript	Reused by update() and loadScenario(); reads sub-index scores, identifies weakest dimension, returns recommendation objects rendered in the UI	Medium

Overall Code Quality Assessment:

Aspect	Rating (1–5)	Comments
Code Layout & Structure	5	Consistent 2-space indentation, separated CSS/JS/HTML files, and logical file naming throughout the project
Readability	5	Self-documenting variable and function names following camelCase; all formula steps commented with UNDP methodology references
Reusability	5	Five core functions are each called in multiple contexts (slider events, scenario loading, export) with zero duplication
Documentation / Comments	4	Block-level JSDoc comments on all functions; inline comments on formula constants. README not yet included — planned for next sprint
Overall Score	**19 / 20**	Excellent code quality across all four dimensions; minor improvement needed in external documentation (README / wiki)